

KIT FAQ Chatbot

Technology stack, architecture, request lifecycle, security, deployment, testing, and file responsibilities

Core design principle

The language model selects approved FAQ records; it never writes the visible institutional answer. The server validates the selection and copies reviewed text from SQLite.

Institution

Kalaignar Karunanidhi Institute of Technology (KIT),
Coimbatore

Application type

Production-ready, fixed-knowledge web chatbot with an
authenticated administration system

Prepared as a clear project and jury reference

Contents

Part I - Project

Purpose, problem, solution, key capabilities, and architecture.

Part II - Technology

Frontend, backend, AI, database, security, deployment, and testing stack.

Part III - Runtime

Complete user-question flow and administrator workflow.

Part IV - Files

Purpose of every source, template, static, data, script, and test file.

Part V - Operations

Routes, production behavior, reliability, strengths, and limitations.

Part VI - Jury

Short presentation speech and recommended live demonstration.

How to use this handbook

Read Parts I-III to understand the system. Use Part IV when the jury asks what a particular file does. Finish with the jury speech and demonstration plan.

1. Project overview

The KIT FAQ Chatbot is a web application that answers student and parent questions about Kalaighnar Karunanidhi Institute of Technology. It covers admissions, programmes, examinations, fees, scholarships, placements, transport, hostel facilities, library services, college rules, and official contact details. Questions may be written in English, Tamil script, or Tanglish.

The problem

A normal generative chatbot may invent a convincing but false fee, date, phone number, eligibility condition, or policy. For institutional communication, that behavior is unacceptable because students may act on the answer.

The solution

This system uses a strict FAQ-only design. Retrieval chooses a small set of approved records, the Groq-hosted model returns only FAQ numbers in JSON, the server validates those numbers, and the final response is copied from stored content. Unknown questions receive one fixed fallback message.

Important distinction

This project is not fine-tuned. Knowledge remains in governed FAQ records. Few-shot prompting teaches selection behavior, while SQLite supplies the final facts.

Supported capabilities

- English, Tamil-script, and Tanglish questions.
- Keyword and TF-IDF-style retrieval of up to eight candidate FAQs.
- Strict JSON-mode FAQ selection through Groq.
- Reviewed Tamil answers when a stored Tamil version exists.
- Fixed fallback for unknown, off-topic, or invalid requests.
- FAQ source display and helpful/not-helpful feedback.
- Persistent administrator-managed FAQ governance with version history and rollback.
- Production health, readiness, metrics, rate limiting, backups, Docker, and CI.

2. Architecture

1	Browser	HTML/CSS/JavaScript accepts a question
2	Flask	Validates JSON, length, origin, host, and rate limit
3	Language	Detects Tamil/Tanglish; translates only for search
4	Retrieval	Selects up to eight published FAQ candidates
5	Groq	Returns JSON with answerable + candidate FAQ IDs
6	Validator	Rejects malformed, duplicate, or out-of-range selection
7	SQLite	Copies approved English or reviewed Tamil answers
8	Response	Stores answer ID, returns text + FAQ sources

Hallucination boundary

Prompt injection or model-generated prose cannot directly enter the visible response. The remaining accuracy risk is selection of an irrelevant existing FAQ, which is measured by the evaluation suite.

3. Technology stack

Frontend HTML5 and Jinja templates create page structure. CSS3 supplies responsive layouts, messages, forms, admin tables, animation, and accessibility. Vanilla JavaScript sends JSON, renders answers, stores local history, creates links, and submits feedback.	Backend Python and Flask implement APIs, templates, security, authentication, retrieval, selection validation, administration, logging, and monitoring. Waitress runs the production WSGI service.
AI The Groq client calls a hosted language model. Few-shot messages and JSON-only instructions restrict the model to choosing numbered FAQ candidates. The model does not write the final institutional response.	Data SQLite in WAL mode stores governed FAQs, version snapshots, unanswered questions, durable answer IDs, feedback, audit events, and migrations. `data/faq.json` is bootstrap seed data only.
Reliability and security Flask-Limiter and optional Redis enforce route limits. Secure sessions, CSRF checks, same-origin rules, host validation, proxy handling, CSP, HSTS, anti-framing, protected metrics, and non-root containers reduce risk.	Delivery and testing Docker and Compose package the web and Redis services. GitHub Actions compiles, tests, audits dependencies, and builds the image. Unittest suites, a 31-case live evaluation, and a bounded load probe verify behavior.

Pinned Python packages

```
Flask==3.1.3
python-dotenv==1.2.3
groq==1.7.0
Flask-Limiter==4.1.1
waitress==3.0.2
redis==6.4.0
```

4. Complete question lifecycle

- 1 The student submits a question through the public form. Pressing Enter or clicking Send triggers one JavaScript submit handler.
- 2 JavaScript sends `POST /chat` with JSON and displays a typing indicator.
- 3 Flask checks JSON shape, required text, length, rate limits, host/origin rules, and service state.
- 4 `language.py` detects Tamil script or common Tanglish words. Translation is generated only to improve English FAQ search.
- 5 `retrieval.py` normalizes words, removes stop words, maps synonyms, applies simple stemming, and ranks published FAQs. At most eight candidates continue.
- 6 If nothing matches, the server skips Groq and immediately returns the fixed fallback.
- 7 `prompt.py` sends strict selector instructions, four role-separated demonstrations, numbered candidates, and the question.
- 8 Groq must return exactly one JSON object containing `answerable` and an array of candidate integer IDs.
- 9 The server rejects extra fields, invalid types, duplicate IDs, contradictions, and out-of-range selections.
- 10 `fact\_check.py` composes the final answer only from selected stored text. Reviewed Tamil is used only when available.
- 11 `storage.py` saves a durable answer ID, question, response, status, and sources in SQLite.
- 12 The browser displays the verified answer, source FAQ questions, safe contact links, and feedback controls.

Why the response is not streamed from the model

The system must validate the complete selector output and copy approved content before anything is shown. The word-by-word effect is a frontend animation after verification, not model-token streaming.

5. Administrator system

The authenticated administration area turns the FAQ data into governed institutional content rather than a static developer-owned file. Administrators sign in at `/admin/login` and every sensitive change is protected and audited.

Administrator capabilities

- Create and preview FAQ records.
- Edit English answers and optional reviewed Tamil answers.
- Assign categories, effective dates, and verification dates.
- Move records through draft, published, and archived states.

- Inspect immutable version snapshots and roll back to an earlier version.
- Validate/import and export the complete FAQ catalogue.
- Review unanswered questions, feedback events, statistics, and administrator audit actions.

Governance rule

Only published records whose effective date has arrived enter retrieval. Draft and archived content cannot be returned to students.

6. File and folder responsibilities

Main application

app.py

Central Flask controller: configuration, security, routes, Groq calls, selector validation, chat, feedback, health, readiness, metrics, admin login, FAQ management, imports/exports, and error handling.

storage.py

SQLite data layer: connection settings, WAL mode, migrations, seed import, FAQ CRUD, version history, rollback, unanswered events, answers, feedback, dashboard statistics, and audit events.

retrieval.py

Search engine: tokenization, stop words, synonym mapping, stemming, weighted document index, TF-IDF-style candidate ranking, and top-k limits.

prompt.py

Defines the fixed fallback, strict selector system prompt, few-shot messages, numbered candidate formatting, and JSON-only selection contract.

language.py

Detects Tamil script and Tanglish, decides whether translation is required, and builds a translation-only prompt.

fact_check.py

Selects stored English/reviewed Tamil text, composes answers from selected FAQs, and verifies exact FAQ-only output.

serve.py

Starts Waitress with configurable host, port, threads, channel timeout, cleanup interval, and server identity.

report.py

Prints administrator summaries of unanswered questions, invalid selector results, feedback, and FAQs needing review.

Public and administrator templates

templates/index.html

Public chatbot structure: header, greeting, conversation area, question form, Send/Clear controls, and static resources.

templates/admin_base.html

Shared admin shell with navigation, page blocks, flash messages, and common resources.

templates/admin_login.html

Administrator authentication form and login status/error presentation.

templates/admin_dashboard.html

FAQ catalogue, operational statistics, unanswered questions, feedback, audit events, and management actions.

templates/admin_faq_form.html

Create/edit/preview form for English/Tamil answers, metadata, status, dates, versions, and rollback.

templates/admin_import.html

JSON catalogue validation, preview, import controls, and error display.

Static browser assets

static/css/style.css

Public chatbot layout, message bubbles, typing animation, sources, feedback, mobile behavior, and reduced-motion support.

static/css/admin.css

Administrator navigation, dashboard, cards, forms, tables, status badges, alerts, buttons, and responsive layout.

static/js/script.js

Public interaction: chat request, loading/error states, safe rendering, linkification, local history, clear action, sources, and feedback.

static/js/admin_login.js

Small login-page behavior while final authentication remains on the server.

Data and configuration

data/faq.json

Bootstrap FAQ catalogue used only when the database initially contains no FAQ records.

data/faq_chatbot.db

Authoritative SQLite database for governed FAQs and persistent events.

data/faq_chatbot.db-wal / .db-shm

SQLite WAL runtime companions for recent writes and shared coordination; not source files.

.env

Private local secrets and runtime configuration. Never commit or display this file.

.env.example

Safe catalogue of every supported environment variable and placeholder.

requirements.txt

Pinned Flask, dotenv, Groq, rate-limiter, Waitress, and Redis dependencies.

.gitignore

Excludes secrets, virtual environments, bytecode, logs, editor state, and runtime artifacts from Git.

.dockerignore

Excludes unnecessary and sensitive paths from the Docker build context.

Deployment and automation

Dockerfile

Builds a non-root production image, installs dependencies, copies source, documents the port, checks health, and starts Waitress.

compose.yaml

Runs one web service plus Redis with environment settings, port mapping, persistent volumes, health checks, and restart policies.

.github/workflows/ci.yml

GitHub Actions pipeline for compilation, unit tests, dependency audit, and container image build.

scripts/create_admin_hash.py

Prompts for an administrator password and produces a secure stored hash.

scripts/backup.py

Uses SQLite's online backup capability to produce a consistent snapshot while the application may be running.

Tests

tests/test_app.py

FAQ data, prompt, page, health, validation, answer, and safe error-response tests.

tests/test_features.py

Retrieval, language detection, FAQ-only composition, sources, feedback, and fallback tests.

tests/test_production.py

Persistence, admin authentication, FAQ governance, security headers, CSRF/origin checks, readiness, metrics, and audit tests.

tests/evaluate.py

Runs the complete live Groq pipeline against exact, paraphrased, unknown, off-topic, and adversarial cases.

tests/test_questions.json

Data-driven evaluation questions with required/forbidden text and fallback rules.

tests/load_test.py

Bounded concurrent deployment probe for response behavior and rate-limit interaction.

7. Main routes

GET	/	Render the public chatbot.
POST	/chat	Validate and answer a student question.
POST	/feedback	Store helpful/not-helpful feedback.
GET	/health	Report basic process liveness.
GET	/ready	Verify database and production configuration readiness.
GET	/metrics	Return bearer-protected operational measurements.
GET/POST	/admin/login	Authenticate an administrator.
GET	/admin	Render the administrator dashboard.
POST	/admin/faq/...	Create, edit, archive, or roll back FAQ records.
GET/POST	/admin/import / export	Validate, replace, or export catalogues.

8. Security and production reliability

Security controls

- Password hashes instead of stored administrator passwords.
- CSRF protection and same-origin checks for administrator changes.
- Secure session cookies, Content Security Policy, anti-framing, HSTS, permissions policy, and no-store API responses.
- Host validation and exact trusted-proxy hop configuration.
- Request-size limits and per-route/global rate limits.
- Bearer protection for metrics and audit logging for administrator actions.
- Non-root Docker process and environment-based secrets.

Reliability controls

- Groq timeout, retry, output-size limit, and circuit breaker.
- SQLite WAL mode, foreign keys, busy timeout, migrations, and online backups.
- Separate liveness and readiness endpoints.
- Structured request logs with request IDs and latency.
- Operational metrics for responses, latency, model errors, circuit events, and fallback rates.
- Redis-capable shared rate limiting and persistent Docker volumes.
- Automated compilation, tests, dependency audit, and image build in CI.

Production scaling boundary

The supported production profile uses one web service with persistent SQLite storage. Before running several web containers, move FAQ and event storage to a network database.

9. Strengths and limitations

Strengths	Limitations
Approved text is copied rather than generated; unknown questions fail safely; prompt injection cannot directly become an answer; FAQ records are governed and versioned; persistent feedback and audit data support improvement; the project includes production security, observability, backups, containers, CI, and tests.	Keyword retrieval is simpler than embedding search; the model can choose an irrelevant existing FAQ; Tamil output needs a reviewed stored translation; information is only as current as administrator verification; SQLite supports the intended single-web-service profile rather than horizontal write scaling.

Why this architecture is appropriate

The application optimizes for institutional correctness and traceability rather than creative conversation. Every displayed fact can be traced to a governed FAQ record and administrator history, which is more important than open-ended generation for admissions and policy questions.

10. Jury presentation guide

Short explanation

Suggested speech

Our project is a production-ready FAQ chatbot for KIT. The frontend uses HTML, CSS, and JavaScript, while Python Flask provides the backend. SQLite stores governed FAQs, answer records, feedback, versions, and audit history. A Groq-hosted model is used only as a strict FAQ selector: it returns candidate FAQ IDs in JSON and never writes the final visible response. After validating the selection, the server copies the approved answer directly from SQLite. The project also includes Tamil and Tanglish support, an authenticated admin dashboard, FAQ rollback, security controls, health and metrics endpoints, backups, Docker deployment, Redis-based rate limiting, automated tests, and GitHub Actions.

Recommended live demonstration

- 1 Ask an exact question such as 'What is the TNEA code?' and point out the displayed FAQ source.
- 2 Ask a paraphrase such as 'When does the library close?' to demonstrate retrieval and model selection.
- 3 Ask a Tanglish or Tamil question to explain translation-for-search and reviewed Tamil output.
- 4 Ask an unknown question such as 'Is there a swimming pool?' to demonstrate safe fallback.
- 5 Try a false premise or prompt injection and explain why model prose cannot enter the response.
- 6 Open the administrator dashboard, edit a draft FAQ, show version history, and explain publication governance.
- 7 Show `/health`, `/ready`, and the protected metrics design to demonstrate production readiness.

Likely jury questions

Why not send all FAQs to the model?

Retrieval reduces token cost, latency, and irrelevant context. Only up to eight high-scoring candidates are sent.

Is the model fine-tuned?

No. Few-shot prompting teaches selection behavior; the governed database supplies knowledge.

How do you stop hallucinations?

The model returns IDs rather than answer prose. The server validates the JSON and copies only approved stored text.

Why use SQLite?

It provides simple persistent relational storage, transactions, version history, and online backups for the intended single-service deployment.

Why use Redis?

Redis allows rate-limit counters to be shared reliably outside the web process in production.

What happens when no answer exists?

The server returns one fixed fallback, logs the question for review, and avoids presenting guessed facts.

End of project handbook